

Collections Framework Concepts with Function, Predicate, Consumer, Supplier, and Producer/Consumer

Java Collections Framework and functional interfaces work together very closely. The **Collections Framework stores and manages data**, while functional interfaces such as Function, Predicate, Consumer, and Supplier define **what operation should be performed on that data**.

A useful mental model is:

```
Collection
↓
List / Set / Map / Queue
↓
Stream
↓
Functional Interface
↓
Function / Predicate / Consumer / Supplier
↓
Result / Action
```

1. Java Collections Framework

Definition

The **Java Collections Framework (JCF)** is a set of interfaces, classes, and utility methods used to store, retrieve, manipulate, and process groups of objects.

The major collection interfaces are:

```
Iterable
|
Collection
|
+-- List
|   +-- ArrayList
|   +-- LinkedList
|
+-- Set
|   +-- HashSet
|   +-- LinkedHashSet
|   +-- TreeSet
|
+-- Queue
    +-- PriorityQueue
    +-- Deque
        +-- ArrayDeque
```

Map is separate from Collection:

```
Map
|
+— HashMap
+— LinkedHashMap
+— TreeMap
+— Hashtable
```

2. Why Functional Interfaces Are Important with Collections

Suppose you have:

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30, 40, 50);
```

You might want to:

1. check whether a number satisfies a condition
2. transform each number
3. print each number
4. generate new values
5. consume values
6. collect the results

Functional interfaces provide the behavior for these operations.

Functional interface	Main method	Input	Output	Typical collection operation
Function<T,R>	apply()	T	R	map()
Predicate<T>	test()	T	boolean	filter()
Consumer<T>	accept()	T	void	forEach()
Supplier<T>	get()	none	T	generate values
UnaryOperator<T>	apply()	T	T	transform same type
BinaryOperator<T>	apply()	T,T	T	combine two values

3. Function<T,R> with Collections

Definition

Function<T,R> represents a function that:

$T \rightarrow R$

It accepts one object and produces another object.

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

Example:

```
Function<Integer, Integer> square =
    n -> n * n;
```

Here:

```
Input  → Integer
Output → Integer
```

Function with List

```
List<Integer> numbers =
    Arrays.asList(1, 2, 3, 4, 5);
```

```
List<Integer> squares =
    numbers.stream()
        .map(n -> n * n)
        .collect(Collectors.toList());
```

```
System.out.println(squares);
```

Output:

```
[1, 4, 9, 16, 25]
```

Execution logic

```
List<Integer>
  ↓
Stream<Integer>
  ↓
map(Function<Integer,Integer>)
  ↓
1 → 1
2 → 4
3 → 9
```

```
4 → 16
5 → 25
↓
List<Integer>
```

The map() method expects a Function.

Conceptually:

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper)
```

Therefore:

```
.map(n -> n * n)
```

is essentially:

```
Function<Integer, Integer> f =
    n -> n * n;

numbers.stream()
    .map(f);
```

4. Function for Converting Objects in a Collection

Suppose:

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}
```

Collection:

```
List<Employee> employees = Arrays.asList(
    new Employee("Amit", 50000),
    new Employee("Rahul", 60000),
    new Employee("Priya", 70000)
);
```

Extract employee names:

```
Function<Employee, String> getName =
```

```
employee -> employee.name;
```

```
List<String> names =  
    employees.stream()  
        .map(getName)  
        .collect(Collectors.toList());
```

Result:

[Amit, Rahul, Priya]

Here:

Employee $\rightarrow$ String

Therefore:

Function<Employee, String>

5. Predicate with Collections

Definition

Predicate<T> represents a condition.

```
@FunctionalInterface  
public interface Predicate<T> {  
    boolean test(T t);  
}
```

It follows:

$T \rightarrow \text{boolean}$

Example:

```
Predicate<Integer> isEven =  
    n -> n % 2 == 0;
```

6. Predicate with filter()

Consider:

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);
```

Find even numbers:

```
List<Integer> evenNumbers =  
    numbers.stream()  
        .filter(n -> n % 2 == 0)  
        .collect(Collectors.toList());
```

Result:

[10, 20, 30]

Conceptually:

10 → true
15 → false
20 → true
25 → false
30 → true

Therefore:

Predicate
↓
test(element)
↓
true / false
↓
filter()

7. Predicate with Objects

```
Predicate<Employee> highSalary =  
    employee -> employee.salary > 60000;
```

Then:

```
List<Employee> result =  
    employees.stream()  
        .filter(highSalary)  
        .collect(Collectors.toList());
```

The flow is:

Employee
↓
Predicate<Employee>
↓
salary > 60000?

↓
true / false
↓
filter

8. Consumer with Collections

Definition

Consumer<T> accepts an object but does not return a value.

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

Its model is:

$T \rightarrow \text{void}$

Example:

```
Consumer<String> print =
    name -> System.out.println(name);
```

9. Consumer with forEach()

```
List<String> names =
    Arrays.asList("Amit", "Rahul", "Priya");

names.forEach(name ->
    System.out.println(name));
```

The `forEach()` method accepts a Consumer.

Conceptually:

```
Consumer<String> consumer =
    name -> System.out.println(name);

names.forEach(consumer);
```

Execution:

```
Amit → accept("Amit") → print
Rahul → accept("Rahul") → print
```

Priya → accept("Priya") → print

10. Consumer for Updating Objects

Suppose:

```
Consumer<Employee> increaseSalary =  
    employee -> employee.salary += 5000;
```

Then:

```
employees.forEach(increaseSalary);
```

The Consumer modifies each employee.

```
Employee  
↓  
Consumer<Employee>  
↓  
salary += 5000  
↓  
void
```

Important:

A Consumer can **modify an object**, but its functional contract does not return a result.

11. Supplier with Collections

Definition

Supplier<T> does not accept an input but produces a value.

```
@FunctionalInterface  
public interface Supplier<T> {  
    T get();  
}
```

Model:

nothing → T

Example:

```
Supplier<String> supplier =
```

```
() -> "Hello Java";
```

Calling:

```
System.out.println(supplier.get());
```

Output:

Hello Java

12. Supplier and Collection Creation

Suppose you need a new ArrayList whenever requested:

```
Supplier<List<String>> listSupplier =  
    () -> new ArrayList<>();
```

Then:

```
List<String> list1 = listSupplier.get();  
List<String> list2 = listSupplier.get();
```

Each call creates a new list.

Supplier

```
|  
+— get() → new ArrayList  
|  
+— get() → new ArrayList  
|  
+— get() → new ArrayList
```

13. Supplier with Stream.generate()

This is an important relationship between Supplier and Collections/Streams.

```
Supplier<Integer> supplier =  
    () -> new Random().nextInt(100);
```

Then:

```
List<Integer> numbers =  
    Stream.generate(supplier)  
        .limit(5)  
        .collect(Collectors.toList());
```

The Supplier produces values whenever the stream requests them.

Conceptually:

```
Stream.generate()
  ↓
Supplier.get()
  ↓
value
  ↓
value
  ↓
value
  ↓
collect()
  ↓
List
```

14. Producer and Consumer Are Different from Supplier and Consumer

This distinction is extremely important.

There are **two different meanings** of "producer" and "consumer" in Java.

Meaning 1 — PECS / Generics

Producer Extends
Consumer Super

This refers to **wildcards and type variance**.

Meaning 2 — Functional interface behavior

A Supplier produces values.

A Consumer consumes values.

These concepts are related, but they are **not the same thing**.

15. Producer in Collections — PECS

Suppose:

```
List<Integer> integers =  
    Arrays.asList(10, 20, 30);
```

A method wants to read numbers:

```
static void printNumbers(  
    List<? extends Number> numbers) {  
  
    for (Number n : numbers) {  
        System.out.println(n);  
    }  
}
```

You can pass:

List<Integer>

because Integer extends Number.

You can also pass:

List<Double>

because Double extends Number.

Therefore:

List<? extends Number>

is a **producer**.

It produces Number values for you to read.

16. Consumer in Collections — PECS

Suppose:

```
List<Number> numbers =  
    new ArrayList<>();
```

You want to add integers:

```
static void addNumbers(  
    List<? super Integer> numbers) {  
  
    numbers.add(10);  
    numbers.add(20);  
    numbers.add(30);  
}
```

This can accept:

List<Integer>
List<Number>
List<Object>

because all are supertypes of Integer.

Therefore:

List<? super Integer>

is a **consumer**.

17. PECS Rule

The fundamental rule is:

PECS

Producer → Extends
Consumer → Super

Producer

If you mainly **read** values:

<? extends T>

Example:

List<? extends Number>

Consumer

If you mainly **write/add** values:

<? super T>

Example:

List<? super Integer>

18. Function<? super T, ? extends R>

Now we reach an important connection between Collections, Streams, and PECS.

The `map()` method is conceptually:

```
<R> Stream<R> map(  
    Function<? super T, ? extends R> mapper)
```

Suppose:

```
Stream<Integer>
```

and:

```
Function<Number, String> converter =  
    number -> "Number: " + number;
```

This can be used because:

```
Integer  
↓  
Number
```

The Function can consume an Integer through its Number parameter.

The result:

```
String
```

is allowed as a subtype of R.

Therefore:

```
Function<? super T, ? extends R>
```

follows PECS on both sides:

```
Function  
|  
+--- input → ? super T  
|  
+--- output → ? extends R
```

19. Predicate<? super T>

`filter()` conceptually accepts:

```
Predicate<? super T>
```

Suppose:

```
Stream<Integer> numbers =  
    Stream.of(10, 20, 30);
```

You can use:

```
Predicate<Number> positive =  
    n -> n.doubleValue() > 0;
```

with:

```
numbers.filter(positive);
```

Why?

Because Predicate is a **consumer of T**.

It receives a value and tests it.

Therefore:

```
Predicate<? super T>
```

makes the API flexible.

20. Consumer<? super T>

forEach() conceptually accepts:

```
Consumer<? super T>
```

Suppose:

```
Consumer<Number> printNumber =  
    n -> System.out.println(n);
```

Then:

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30);
```

```
numbers.forEach(printNumber);
```

This works because:

Integer → Number

The Consumer can consume an Integer as a Number.

21. Supplier<T> Is Different

A Supplier does not consume a collection element.

It produces a value.

```
Supplier<Integer> supplier =  
    () -> 100;
```

Model:

Supplier<Integer>

input: nothing

output: Integer

Therefore, Supplier generally does not use:

Supplier<? super T>

for its return value.

A Supplier's important direction is **output**.

22. Complete Collection Pipeline

Consider:

```
List<Employee> employees = Arrays.asList(  
    new Employee("Amit", 50000),  
    new Employee("Rahul", 70000),  
    new Employee("Priya", 80000)  
);
```

We want:

1. select employees with salary > 60000
2. extract names
3. print names

Step 1 — Predicate

```
Predicate<Employee> highSalary =  
    employee -> employee.salary > 60000;
```

Step 2 — Function

```
Function<Employee, String> getName =  
    employee -> employee.name;
```

Step 3 — Consumer

```
Consumer<String> print =  
    name -> System.out.println(name);
```

Step 4 — Pipeline

```
employees.stream()  
    .filter(highSalary)  
    .map(getName)  
    .forEach(print);
```

Execution:

```
Employee Collection  
  |  
  v  
stream()  
  |  
  v  
Predicate<Employee>  
  |  
  | salary > 60000  
  v  
filtered employees  
  |  
  v  
Function<Employee, String>  
  |  
  | employee.name  
  v  
Names  
  |  
  v  
Consumer<String>  
  |  
  | print  
  v  
Output
```

23. One Pipeline Showing All Four Interfaces

```
List<Employee> employees = Arrays.asList(
    new Employee("Amit", 50000),
    new Employee("Rahul", 70000),
    new Employee("Priya", 80000)
);

Predicate<Employee> eligible =
    e -> e.salary >= 60000;

Function<Employee, String> nameFunction =
    e -> e.name;

Consumer<String> output =
    name -> System.out.println("Employee: " + name);

employees.stream()
    .filter(eligible)
    .map(nameFunction)
    .forEach(output);
```

Conceptually:

```
Collection
↓
Stream
↓
Predicate
↓
filter()
↓
Function
↓
map()
↓
Consumer
↓
forEach()
```

24. Where Does Supplier Fit?

Supplier generally sits at the **source/generation side**.

Example:

```
Supplier<Employee> employeeSupplier =  
    () -> new Employee("New Employee", 50000);
```

Generate employees:

```
List<Employee> employees =  
    Stream.generate(employeeSupplier)  
        .limit(5)  
        .collect(Collectors.toList());
```

Then process them:

```
employees.stream()  
    .filter(e -> e.salary > 40000)  
    .map(e -> e.name)  
    .forEach(System.out::println);
```

Overall:

```
Supplier  
↓  
generate values  
↓  
Collection / Stream  
↓  
Predicate  
↓  
filter  
↓  
Function  
↓  
map  
↓  
Consumer  
↓  
forEach
```

25. Collection.forEach() vs Stream.forEach()

Both use Consumer.

Collection

```
List<String> names =  
    Arrays.asList("Amit", "Rahul", "Priya");  
  
names.forEach(  
    name -> System.out.println(name)
```

```
);
```

Stream

```
names.stream()  
    .forEach(  
        name -> System.out.println(name)  
    );
```

Both conceptually use:

Consumer<? super T>

The difference is that Stream allows intermediate processing:

```
names.stream()  
    .filter(...)  
    .map(...)  
    .sorted(...)  
    .forEach(...);
```

26. Collections + Functional Interfaces + Method References

Instead of:

```
names.forEach(  
    name -> System.out.println(name)  
);
```

you can use:

```
names.forEach(System.out::println);
```

Here:

```
System.out::println
```

is compatible with:

```
Consumer<String>
```

Similarly:

```
employees.stream()  
    .map(Employee::getName)  
    .forEach(System.out::println);
```

The conceptual types are:

Employee::getName



Function<Employee, String>

System.out::println



Consumer<String>

27. removeIf() Uses Predicate

Collections also directly use functional interfaces.

Example:

```
List<Integer> numbers =  
    new ArrayList<>(  
        Arrays.asList(10, 15, 20, 25, 30)  
    );
```

```
numbers.removeIf(n -> n % 2 != 0);
```

Result:

```
[10, 20, 30]
```

removeIf() uses a Predicate.

Conceptually:

```
boolean removeIf(Predicate<? super E> filter)
```

Therefore:

Collection



removeIf()



Predicate



true → remove

false → keep

28. replaceAll() Uses UnaryOperator

Collections also use another functional interface:

UnaryOperator<T>

Example:

```
List<String> names =  
    new ArrayList<>(  
        Arrays.asList("amit", "rahul", "priya")  
    );
```

```
names.replaceAll(  
    name -> name.toUpperCase()  
);
```

Result:

[AMIT, RAHUL, PRIYA]

Conceptually:

String → String

That's why:

UnaryOperator<String>

is appropriate.

29. sort() Uses Comparator

Another important functional interface used by collections is:

Comparator<T>

Example:

```
List<Integer> numbers =  
    new ArrayList<>(  
        Arrays.asList(50, 10, 40, 20)  
    );
```

```
numbers.sort(  
    (a, b) -> Integer.compare(a, b)  
);
```

Or:

```
numbers.sort(Integer::compare);
```

Comparator is effectively:

$$(T, T) \rightarrow \text{int}$$

It compares two objects.

30. Important Collection Methods and Functional Interfaces

| Collection/Stream method | Functional interface | Purpose |
|--------------------------|--------------------------------------|----------------------------|
| filter() | Predicate<T> | Select elements |
| map() | Function<T,R> | Transform elements |
| forEach() | Consumer<T> | Perform action |
| removeIf() | Predicate<T> | Remove matching elements |
| replaceAll() | UnaryOperator<T> | Replace/transform elements |
| sort() | Comparator<T> | Sort elements |
| Stream.generate() | Supplier<T> | Generate elements |
| Stream.iterate() | UnaryOperator<T> | Generate next element |
| reduce() | BinaryOperator<T> /
related forms | Combine elements |
| collect() | Collector | Accumulate results |

31. Map and Functional Interfaces

The Map interface has many functional methods.

forEach()

```
Map<Integer, String> employees =  
    new HashMap<>();
```

```
employees.put(101, "Amit");  
employees.put(102, "Rahul");
```

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " " + name)  
);
```

Conceptually:

`BiConsumer<Integer, String>`

because there are two inputs:

`Integer + String → void`

32. `Map.computeIfAbsent()` and Supplier-like Behavior

Example:

```
Map<String, List<String>> map =  
    new HashMap<>();
```

```
map.computeIfAbsent(  
    "Java",  
    key -> new ArrayList<>()  
);
```

The second argument is:

`Function<? super K, ? extends V>`

Notice that it is **not** a Supplier, because the function receives the key.

Conceptually:

`key → new value`

rather than:

`nothing → new value`

This distinction is important.

33. `Map.replaceAll()`

```
Map<Integer, String> map =
```

```
new HashMap<>();

map.put(1, "amit");
map.put(2, "rahul");

map.replaceAll(
    (key, value) -> value.toUpperCase()
);
```

Conceptually:

`BiFunction<K,V,V>`

because it receives:

key + value

and returns a new value.

34. The Most Important Mental Model

Think about the direction of data.

Function

$T \longrightarrow R$

Transform.

Example:

Employee $\rightarrow$ String

Predicate

$T \longrightarrow \text{boolean}$

Check.

Example:

Employee $\rightarrow$ true/false

Consumer

T —————→ void

Use/consume.

Example:

Employee → print employee

Supplier

nothing ———→ T

Produce.

Example:

nothing → Employee

Producer wildcard

? extends T

Means:

I can READ T values.

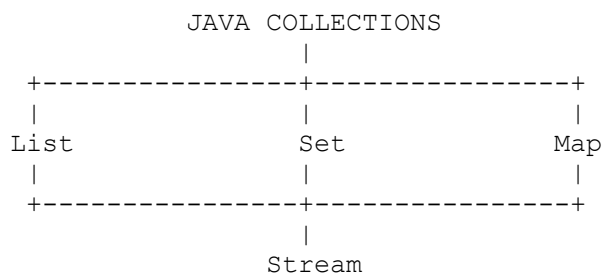
Consumer wildcard

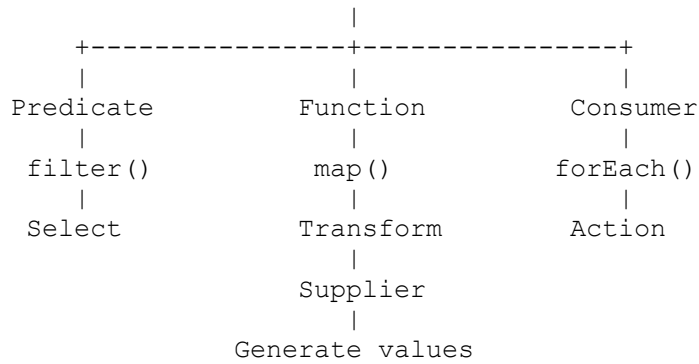
? super T

Means:

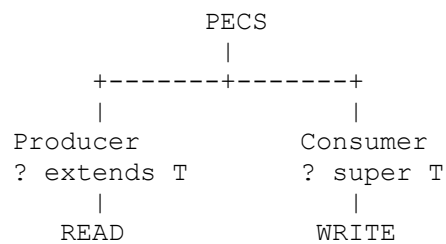
I can WRITE T values.

35. Complete Relationship





And with generics:



36. Real-World Example — Employee Processing

Suppose an application has:

List<Employee> employees;

Requirement:

Find employees whose salary is greater than ₹60,000, extract their names, convert names to uppercase, and print them.

Predicate

```
Predicate<Employee> salaryCheck =
    e -> e.salary > 60000;
```

Function 1

```
Function<Employee, String> getName =
    e -> e.name;
```

Function 2

```
Function<String, String> uppercase =
    String::toUpperCase;
```

Consumer

```
Consumer<String> print =  
    System.out::println;
```

Pipeline

```
employees.stream()  
    .filter(salaryCheck)  
    .map(getName)  
    .map(uppercase)  
    .forEach(print);
```

Execution:

```
Employee  
↓  
Predicate  
salary > 60000?  
↓  
Employee  
↓  
Function  
Employee → String  
↓  
Name  
↓  
Function  
String → String  
↓  
Uppercase Name  
↓  
Consumer  
↓  
Print
```

37. Interview-Level Summary

When you see:

List<T>

think:

Where is my data stored?

When you see:

Predicate<T>

think:

Should this element be selected?

When you see:

Function<T,R>

think:

How do I transform this element?

When you see:

Consumer<T>

think:

What action should I perform on this element?

When you see:

Supplier<T>

think:

How can I obtain/create a value without an input?

When you see:

? extends T

think:

Producer — I mainly read values.

When you see:

? super T

think:

Consumer — I mainly write/add values.

Final Comparison

| Concept | Syntax | Direction | Main purpose | Common example |
|--------------------------------|-----------------------------|--|--------------|---------------------------|
| Function<T,R> | $T \rightarrow R$ | Input $\rightarrow$ Output | Transform | map() |
| Predicate<T> | $T \rightarrow$
boolean | Input $\rightarrow$
decision | Test/filter | filter() |
| Consumer<T> | $T \rightarrow$ void | Input $\rightarrow$ action | Process/use | forEach() |
| Supplier<T> | $() \rightarrow T$ | No input $\rightarrow$
output | Generate | generate() |
| Producer | ? extends T | Collection $\rightarrow$
reader | Read | List<? extends
Number> |
| Consumer | ? super T | writer $\rightarrow$
collection | Write | List<? super Integer> |
| UnaryOperator<T> | $T \rightarrow T$ | Same type | Transform | replaceAll() |
| Comparator<T> | $(T,T) \rightarrow$ int | Two inputs $\rightarrow$
comparison | Sort | sort() |
| BiFunction<T,U,R> | $(T,U) \rightarrow R$ | Two inputs $\rightarrow$
output | Transform | Map.compute... |
| BiConsumer<T,U> | $(T,U) \rightarrow$
void | Two inputs $\rightarrow$
action | Process | Map.forEach() |

The key relationship

COLLECTIONS = DATA
FUNCTION = TRANSFORMATION
PREDICATE = DECISION
CONSUMER = ACTION
SUPPLIER = CREATION
PRODUCER = READ (? extends)
CONSUMER = WRITE (? super)
STREAM = PROCESSING PIPELINE

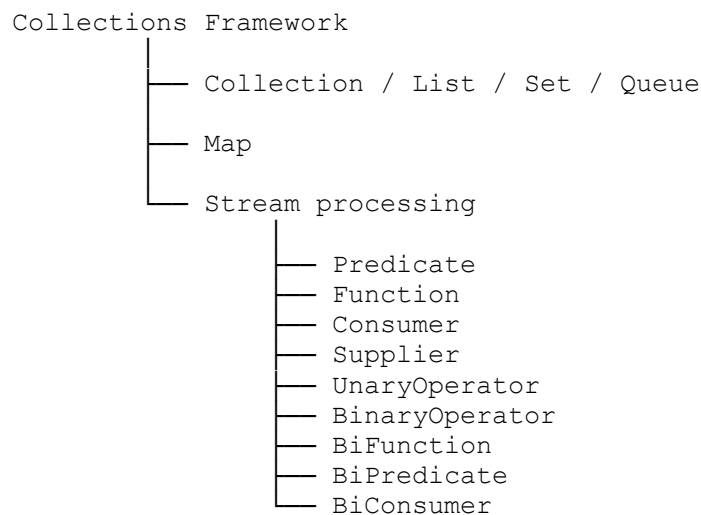
This is the central connection between the **Collections Framework, Generics/PECS, Streams, and Java functional interfaces.**

Yes. There is an important distinction in your statement:

The Collections Framework classes do not "implement" Function, Predicate, Consumer, or Supplier as their primary interfaces.

Rather, many **Collection Framework classes and methods accept, return, or work together with these functional interfaces**—especially through Streams, default methods, Map methods, and collection-processing APIs.

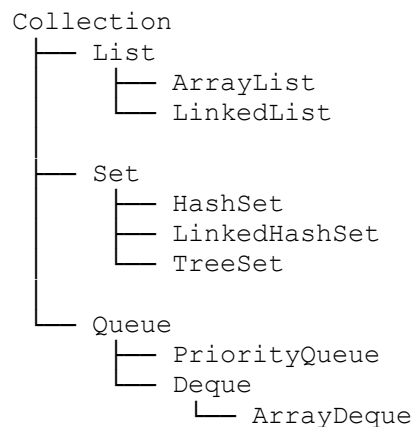
So the correct relationship is:

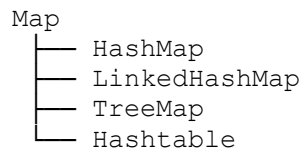


Below is the complete mapping.

1. Collections Framework and Functional Interfaces

The Java Collections Framework provides data structures such as:





Functional interfaces provide behavior for processing the data.

2. Four Fundamental Functional Interfaces

The easiest way to understand the relationship is:

| Functional interface | Meaning | Input | Output |
|----------------------------|----------------|-------|---------|
| Function<T,R> | Transform | 1 | 1 |
| Predicate<T> | Test | 1 | boolean |
| Consumer<T> | Perform action | 1 | void |
| Supplier<T> | Produce value | 0 | 1 |

Think of them as:

Function → CHANGE

Predicate → CHECK

Consumer → USE

Supplier → CREATE

3. Function<T,R> in Collections

Definition

Function<T,R> represents a transformation:

$T \rightarrow R$

Its abstract method is:

R apply(T t);

Example:

```
Function<String, Integer> length =  
    s -> s.length();
```

It converts:

String → Integer

4. Function with List

Consider:

```
List<String> names =  
    Arrays.asList("Amit", "Rahul", "Priya");
```

Convert every name to its length:

```
List<Integer> lengths =  
    names.stream()  
        .map(name -> name.length())  
        .collect(Collectors.toList());
```

Output:

[4, 5, 5]

The lambda:

name -> name.length()

is a:

Function<String, Integer>

Conceptually:

```
List<String>  
  ↓  
Stream<String>  
  ↓  
Function<String,Integer>  
  ↓  
map()  
  ↓  
Stream<Integer>  
  ↓  
List<Integer>
```

5. Function with Employee Objects

```
class Employee {  
    private String name;  
    private double salary;  
  
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
}
```

Collection:

```
List<Employee> employees = Arrays.asList(  
    new Employee("Amit", 50000),  
    new Employee("Rahul", 70000),  
    new Employee("Priya", 80000)  
);
```

Extract names:

```
Function<Employee, String> getName =  
    Employee::getName;
```

```
List<String> names =  
    employees.stream()  
        .map(getName)  
        .collect(Collectors.toList());
```

Here:

Employee → String

Therefore:

```
Function<Employee, String>
```

6. Predicate<T> in Collections

Definition

Predicate<T> represents a condition.

```
boolean test(T t);
```

Its model is:

$T \rightarrow \text{boolean}$

Example:

```
Predicate<Integer> even =  
    n -> n % 2 == 0;
```

7. Predicate with filter()

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);
```

Filter even numbers:

```
List<Integer> result =  
    numbers.stream()  
        .filter(n -> n % 2 == 0)  
        .collect(Collectors.toList());
```

Result:

```
[10, 20, 30]
```

The lambda:

```
n -> n % 2 == 0
```

is:

```
Predicate<Integer>
```

Execution:

```
10 → true → keep  
15 → false → discard  
20 → true → keep  
25 → false → discard  
30 → true → keep
```

8. Predicate with Collection's removeIf()

This is particularly important because Predicate isn't restricted to Streams.

```
List<Integer> numbers =  
    new ArrayList<>(  
        Arrays.asList(10, 15, 20, 25, 30)  
    );
```

```
numbers.removeIf(n -> n % 2 != 0);
```

Result:

```
[10, 20, 30]
```

Conceptually, removeIf() takes:

```
Predicate<? super E>
```

Therefore:

```
Collection  
↓  
removeIf()  
↓  
Predicate  
↓  
true → remove  
false → retain
```

9. Predicate with Objects

```
Predicate<Employee> highSalary =  
    employee -> employee.getSalary() > 60000;
```

Use it:

```
List<Employee> result =  
    employees.stream()  
        .filter(highSalary)  
        .collect(Collectors.toList());
```

The relationship is:

```
Employee  
↓  
Predicate<Employee>  
↓
```

salary > 60000?

↓

true / false

10. Consumer<T> in Collections

Definition

Consumer<T> accepts an object and performs an operation.

void accept(T t);

Model:

$T \rightarrow \text{void}$

Example:

```
Consumer<String> print =  
    name -> System.out.println(name);
```

11. Consumer with forEach()

```
List<String> names =  
    Arrays.asList("Amit", "Rahul", "Priya");
```

```
names.forEach(  
    name -> System.out.println(name)  
);
```

The lambda is:

Consumer<String>

because:

$\text{String} \rightarrow \text{void}$

You can also write:

```
Consumer<String> print =  
    System.out::println;
```

```
names.forEach(print);
```

12. Consumer with Stream

```
employees.stream()
    .forEach(
        employee ->
            System.out.println(employee.getName())
    );
```

The Consumer receives one Employee:

Employee → void

Therefore:

Consumer<Employee>

13. Consumer Can Modify Objects

A Consumer does not return a result, but it can modify a mutable object.

```
Consumer<Employee> increaseSalary =
    e -> {
        // example of mutating a mutable Employee
        // e.setSalary(e.getSalary() + 5000);
    };
```

Conceptually:

```
Employee
↓
Consumer<Employee>
↓
perform action
↓
void
```

14. Supplier<T> and Collections

Definition

Supplier<T> produces a value without accepting an input.

T get();

Model:

nothing $\rightarrow$ T

Example:

```
Supplier<String> supplier =  
    () -> "Hello";
```

Calling:

```
String value = supplier.get();
```

produces:

Hello

15. Supplier and Collection Creation

Suppose you want a factory for new lists:

```
Supplier<List<String>> listSupplier =  
    ArrayList::new;
```

Then:

```
List<String> list1 = listSupplier.get();  
List<String> list2 = listSupplier.get();
```

Every call can create a new ArrayList.

Relationship:

```
Supplier<List<String>>  
  ↓  
  get()  
  ↓  
  new ArrayList<>()
```

16. Supplier with Stream.generate()

This is one of the clearest examples of a Supplier producing collection elements.

```
Supplier<Integer> supplier =  
    () -> new Random().nextInt(100);
```

Then:

```
List<Integer> numbers =  
    Stream.generate(supplier)  
        .limit(5)  
        .collect(Collectors.toList());
```

The flow is:

```
Supplier  
↓  
get()  
↓  
Integer  
↓  
Stream  
↓  
collect()  
↓  
List<Integer>
```

The Supplier is the **source of the values**.

17. The Four Interfaces Together

Suppose:

```
List<Employee> employees = ...
```

Requirement:

Find employees earning more than ₹60,000, get their names, and print them.

Predicate

```
Predicate<Employee> highSalary =  
    e -> e.getSalary() > 60000;
```

Function

```
Function<Employee, String> name =  
    Employee::getName;
```

Consumer

```
Consumer<String> print =  
    System.out::println;
```

Pipeline

```
employees.stream()  
    .filter(highSalary)  
    .map(name)  
    .forEach(print);
```

The entire algorithm is:

Collection



Stream



Predicate



filter()



Function



map()



Consumer



forEach()



Output

18. Where Supplier Fits in the Same Architecture

Suppose employees have to be generated first.

```
Supplier<Employee> employeeSupplier =  
    () -> new Employee("Unknown", 50000);
```

Then:

```
List<Employee> employees =  
    Stream.generate(employeeSupplier)  
        .limit(5)  
        .collect(Collectors.toList());
```

Now:

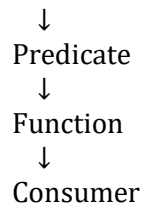
Supplier



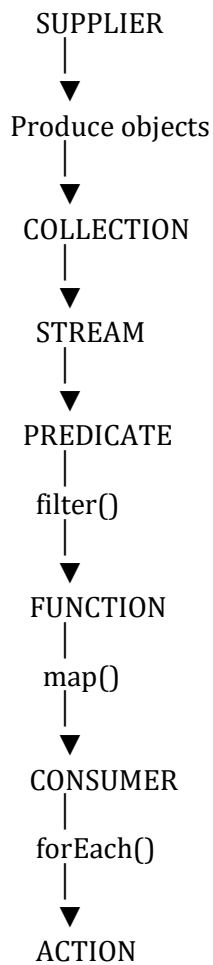
generate data



Collection



So a complete functional architecture can be:



19. Producer Is a Different Concept

This is where terminology can become confusing.

When you say **producer**, there are two different concepts.

Functional programming producer

A `Supplier<T>` is commonly described as a producer:

$() \rightarrow T$

Generics producer

A wildcard such as:

? extends T

is called a producer under the **PECS** rule.

These are related ideas but are not identical.

20. Producer — ? extends T

Suppose:

```
List<Integer> integers =  
    Arrays.asList(10, 20, 30);
```

Method:

```
static void printNumbers(  
    List<? extends Number> numbers) {  
  
    for (Number n : numbers) {  
        System.out.println(n);  
    }  
}
```

Call:

```
printNumbers(integers);
```

Why does it work?

Because:

Integer extends Number

The collection **produces values for reading**.

Therefore:

```
List<? extends Number>
```

is a producer.

21. Consumer — ? super T

Now:

```
static void addNumbers(  
    List<? super Integer> numbers) {  
  
    numbers.add(10);  
    numbers.add(20);  
}
```

This can accept:

```
List<Integer>  
List<Number>  
List<Object>
```

because each is a valid supertype of Integer.

This is a **consumer** because the collection receives Integer values.

22. PECS

The famous rule is:

```
P → Producer  
E → Extends
```

```
C → Consumer  
S → Super
```

Therefore:

```
Producer → ? extends T  
Consumer → ? super T
```

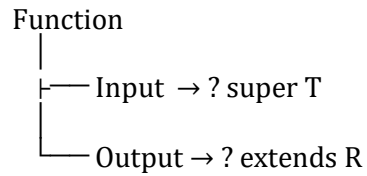
23. Function<? super T, ? extends R>

This is one of the most important collection/Stream concepts.

map() accepts a Function conceptually like:

```
Function<? super T, ? extends R>
```

There are two directions:



Why?

Because the Function **consumes** its input:

$T \rightarrow \text{Function}$

and **produces** its output:

$\text{Function} \rightarrow R$

So:

Function
input = consumer side
output = producer side

24. Example of Function PECS

Suppose:

```
Stream<Integer> stream =  
    Stream.of(10, 20, 30);
```

You can use:

```
Function<Number, String> converter =  
    n -> "Value = " + n;
```

Then:

```
stream.map(converter)  
    .forEach(System.out::println);
```

Why can a `Function<Number,String>` work with Integer values?

Because:

Integer
↓

Number

The Function accepts a supertype of the stream element.

25. Predicate<? super T>

filter() conceptually accepts:

Predicate<? super T>

Suppose:

```
Predicate<Number> positive =  
    n -> n.doubleValue() > 0;
```

Then:

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30);  
  
numbers.stream()  
    .filter(positive)  
    .forEach(System.out::println);
```

Why?

Because Predicate consumes an element:

```
Integer  
↓  
Predicate<Number>  
↓  
boolean
```

Therefore:

Predicate<? super T>

26. Consumer<? super T>

forEach() conceptually uses:

Consumer<? super T>

Example:

```
Consumer<Number> print =  
    n -> System.out.println(n);
```

```
List<Integer> numbers =  
    Arrays.asList(10, 20, 30);
```

```
numbers.forEach(print);
```

Again:

```
Integer  
↓  
Number  
↓  
Consumer
```

The Consumer accepts the Integer as a Number.

27. Why Supplier Is Different

Supplier produces the result:

```
Supplier<Integer>
```

means:

nothing $\rightarrow$ Integer

It does not consume a collection element.

Compare:

```
Predicate<T>  
T  $\rightarrow$  boolean
```

```
Function<T,R>  
T  $\rightarrow$  R
```

```
Consumer<T>  
T  $\rightarrow$  void
```

```
Supplier<T>  
()  $\rightarrow$  T
```

This is the most important functional-interface classification.

28. Collections Classes and Their Functional-Interface Relationships

Here is the broader mapping.

| Collection class/interface | Important functional relationship |
|----------------------------|--|
| ArrayList | forEach(), removeIf(), replaceAll(), sort() |
| LinkedList | Same Collection/List functional operations |
| HashSet | forEach(), removeIf() |
| LinkedHashSet | forEach(), removeIf() |
| TreeSet | forEach(), removeIf() |
| PriorityQueue | removeIf(), forEach() |
| ArrayDeque | removeIf(), forEach() |
| HashMap | forEach(), replaceAll(), compute(), computeIfAbsent(), etc. |
| LinkedHashMap | Same Map functional operations |
| TreeMap | Same Map functional operations |
| Collections | Utility operations, including algorithms using Comparator |
| Arrays | Array processing, often used with Streams |
| Stream | Extensive use of Predicate, Function, Consumer, Supplier, etc. |

The key point is that the **classes themselves generally don't implement these functional interfaces**.

Instead, their **methods accept functional-interface implementations**.

29. ArrayList Example

```
List<String> names =  
    new ArrayList<>();  
  
names.add("Amit");
```

```
names.add("Rahul");
names.add("Priya");
```

Consumer

```
names.forEach(System.out::println);
```

Predicate

```
names.removeIf(
    name -> name.startsWith("A")
);
```

UnaryOperator

```
names.replaceAll(
    String::toUpperCase
);
```

So:

```
ArrayList
|
├── forEach()   → Consumer
├── removeIf() → Predicate
└── replaceAll() → UnaryOperator
```

30. HashSet Example

```
Set<Integer> numbers =
    new HashSet<>{
        Arrays.asList(10, 20, 30, 40)
    };
```

Consumer:

```
numbers.forEach(
    n -> System.out.println(n)
);
```

Predicate:

```
numbers.removeIf(
    n -> n < 30
);
```

The Set itself doesn't become a Predicate or Consumer.

Instead:

HashSet

↓

method accepts functional interface

↓

lambda supplies behavior

31. HashMap Example

```
Map<Integer, String> employees =  
    new HashMap<>();
```

```
employees.put(101, "Amit");  
employees.put(102, "Rahul");  
employees.put(103, "Priya");
```

forEach()

```
employees.forEach(  
    (id, name) ->  
        System.out.println(id + " " + name)  
);
```

This is:

BiConsumer<Integer, String>

because:

Integer + String → void

32. Map.replaceAll()

```
employees.replaceAll(  
    (id, name) -> name.toUpperCase()  
);
```

This is conceptually:

BiFunction<Integer, String, String>

because:

key + value → new value

33. Map.computeIfAbsent()

```
Map<String, List<String>> departments =  
    new HashMap<>();
```

```
departments.computeIfAbsent(  
    "IT",  
    key -> new ArrayList<>()  
);
```

The lambda is a:

Function<String, List<String>>

not a Supplier.

Why?

Because it receives the key:

key → List

Whereas Supplier would be:

nothing → List

This is a very important distinction.

34. Map.computeIfPresent()

```
departments.computeIfPresent(  
    "IT",  
    (key, value) -> {  
        value.add("Java");  
        return value;  
    }  
);
```

This uses a:

BiFunction<K,V,V>

Conceptually:

key + currentValue
↓

newValue

35. Collection + Stream + Four Interfaces

A real-world example:

```
List<Employee> employees = ...
```

We want:

Find employees earning above ₹60,000, convert their names to uppercase, and print them.

```
Predicate<Employee> salaryPredicate =  
    e -> e.getSalary() > 60000;
```

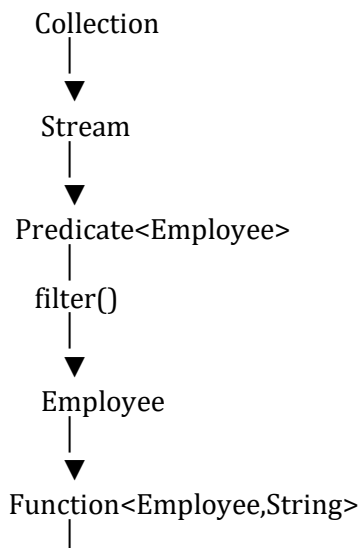
```
Function<Employee, String> getName =  
    Employee::getName;
```

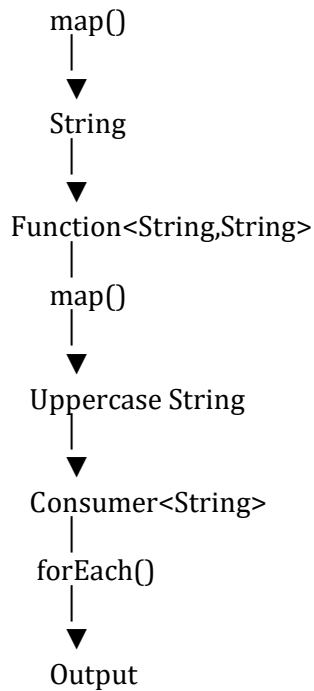
```
Function<String, String> upperCase =  
    String::toUpperCase;
```

```
Consumer<String> print =  
    System.out::println;
```

```
employees.stream()  
    .filter(salaryPredicate)  
    .map(getName)  
    .map(upperCase)  
    .forEach(print);
```

Execution:





36. The Most Important Correction

It is not accurate to say:

"ArrayList, HashSet, HashMap, etc. implement Function, Predicate, Consumer, and Supplier."

Instead, say:

The Collections Framework integrates with functional interfaces by accepting them as method parameters, returning functional objects in some APIs, and using them internally for operations such as filtering, transformation, iteration, generation, comparison, computation, and replacement.

For example:

```
ArrayList
|
├── forEach(Consumer)
├── removeIf(Predicate)
├── replaceAll(UnaryOperator)
└── sort(Comparator)
```

And:

```
Stream
|
├── filter(Predicate)
```

- └─ map(Function)
- └─ forEach(Consumer)
- └─ generate(Supplier)
- └─ iterate(UnaryOperator)
- └─ reduce(BinaryOperator)

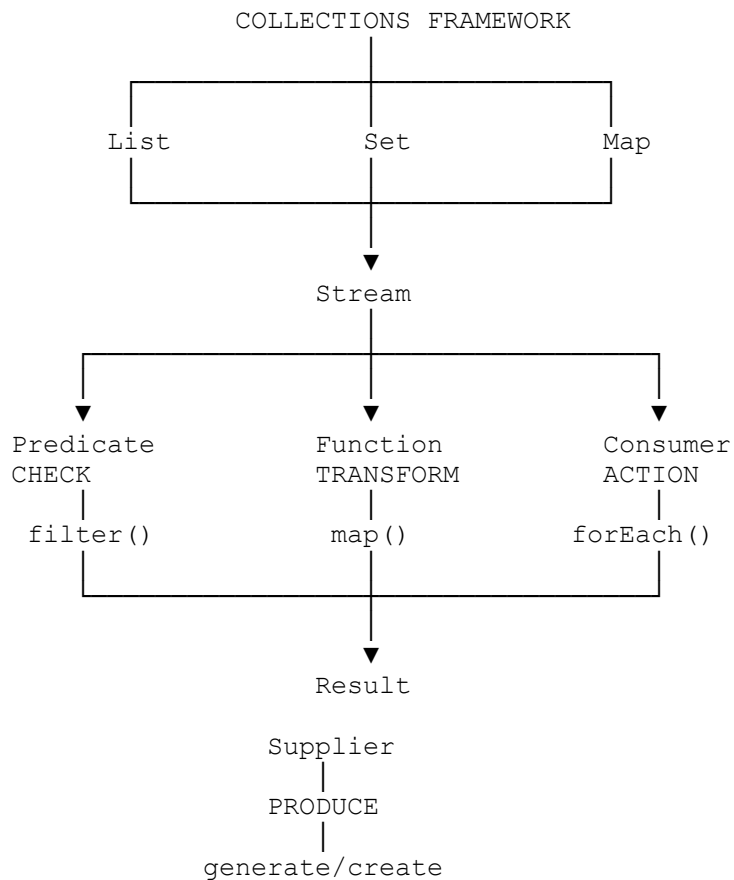
And:

Map

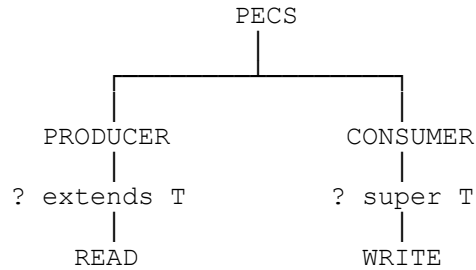
- └─ forEach(BiConsumer)
- └─ replaceAll(BiFunction)
- └─ compute(BiFunction)
- └─ computeIfAbsent(Function)
- └─ computeIfPresent(BiFunction)
- └─ remove(key,value) / replace(key,value,value)

37. Final Architecture

The relationship can be summarized as:



And on the **generic variance** side:



One-line memory trick

| | |
|------------|-------------------|
| Supplier | → produces data |
| Collection | → stores data |
| Predicate | → selects data |
| Function | → transforms data |
| Consumer | → processes data |

So the typical functional collection pipeline is:

```
Supplier
↓
produce
↓
Collection / Stream
↓
Predicate
↓
filter
↓
Function
↓
map
↓
Consumer
↓
forEach
↓
action/output
```

This distinction between **"a collection class implements a functional interface"** and **"a collection class has methods that accept functional interfaces"** is especially important for Java interviews and for understanding the actual design of the Collections Framework.